1. Problem Statement & Objectives

Problem Statement

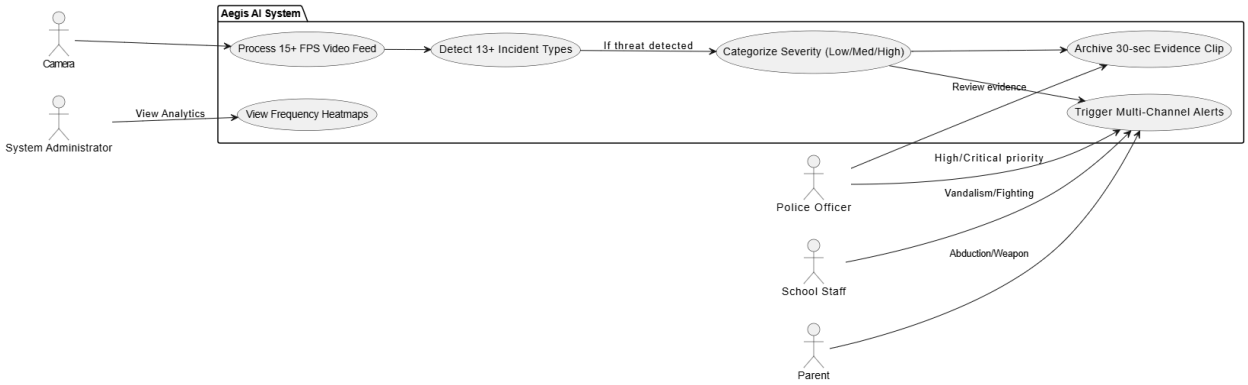
Traditional security setups require 24/7 human observation, which is prone to fatigue and often fails to prevent incidents in real-time. Stakeholders—including police forces, schools, and parents—require a proactive system that automatically flags critical incidents (like shootings, explosions, or vandalism) to dispatch emergency units or trigger lockdowns without waiting for manual intervention.

Project Objectives

- **Active Monitoring:** Transform passive surveillance into an active system using Spatial-Temporal neural networks to detect 13 specific incidents.
- **Multi-Tiered Alerting:** Provide immediate, multi-channel notifications (SMS, Email, Dashboard Push) to the correct stakeholders based on priority.
- **Evidence Collection:** Automatically archive 30-second video clips of incidents to serve as reliable evidence for legal or administrative prosecution.

Use Case Diagram & Descriptions

- **System Actors:** Security Personnel, System Administrators, Police Officers, School Staff, Students, and Parents.
- **Key Interactions:**
    - *Police:* Receive critical alerts, view live feeds and precise GPS locations, dispatch tactical units, and review 30-second evidence clips.
    - *Schools:* Detect fighting/vandalism, categorize severity, trigger mobile/desktop alerts, and initiate emergency protocols.
    - *Parents:* Receive emergency alerts (abduction, weapon detection, critical injury) and verify child status.



Functional Requirements (FRs)

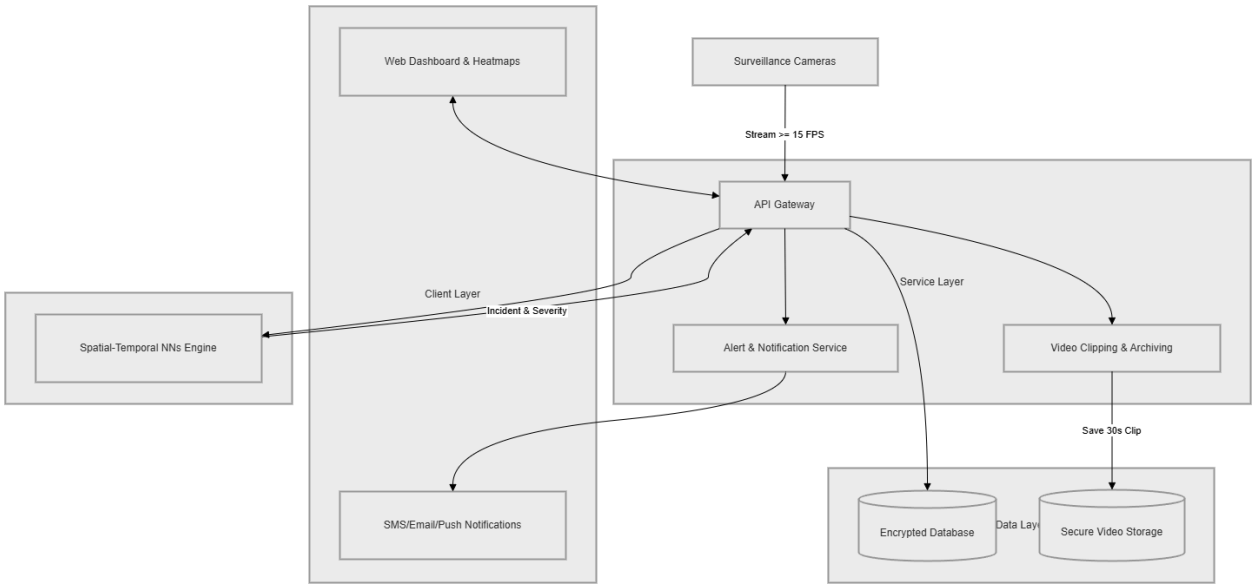
- **FR-1 (Real-time Detection):** Detect 13+ specific incidents (Abuse, Arson, Robbery, etc.) within seconds.
- **FR-2 (Severity Categorization):** Automatically label incidents as Low, Medium, or High priority.
- **FR-3 (Alert Triggering):** Send notifications via SMS, Email, and Dashboard Push to designated stakeholders.
- **FR-4 (Analytics):** Provide a web-based UI for live feeds, logs, and frequency heatmaps.
- **FR-5 (Archiving):** Automatically clip and store 30 seconds of video prior to and after a detected incident.

Non-Functional Requirements (NFRs)

- **Performance:** Process video frames at a minimum of 15 FPS to ensure real-time accuracy.
- **Security:** Encrypt all video data to prevent unauthorized access or tampering.
- **Reliability:** Maintain high uptime for life-critical event detection.
- **Scalability:** Support multiple camera feeds without a drop in processing speed.

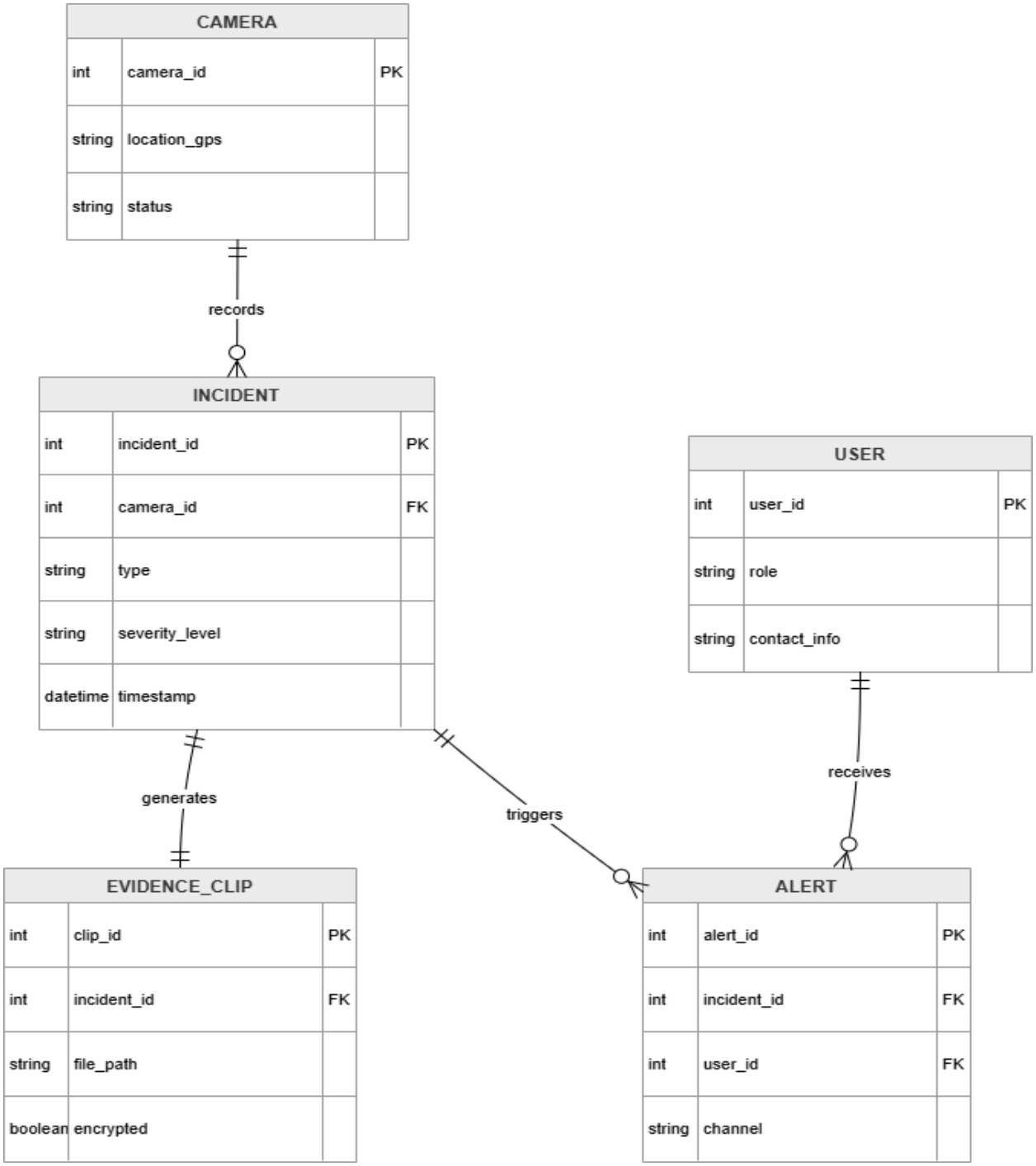
Software Architecture

The system adopts an API-Driven Microservices architecture to handle the distinct loads of video streaming, AI inference, and multi-channel alerting.



2. Database Design & Data Modeling

Entity-Relationship Diagram (ERD)



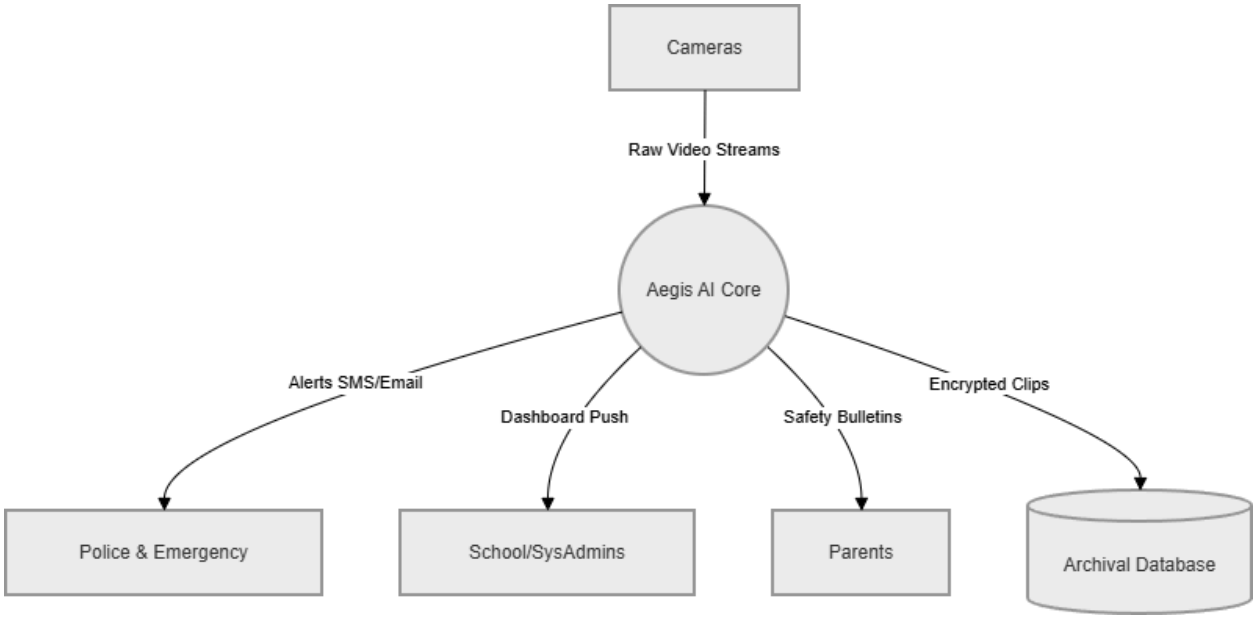
Logical & Physical Schema Considerations

- **Normalization:** Tables are designed to 3NF to prevent data redundancy.
- **Attributes:** The INCIDENT table includes a severity_level (Low, Med, High) based on FR-2. The EVIDENCE_CLIP table includes paths to secure blob storage for the 30-second clips based on FR-5.

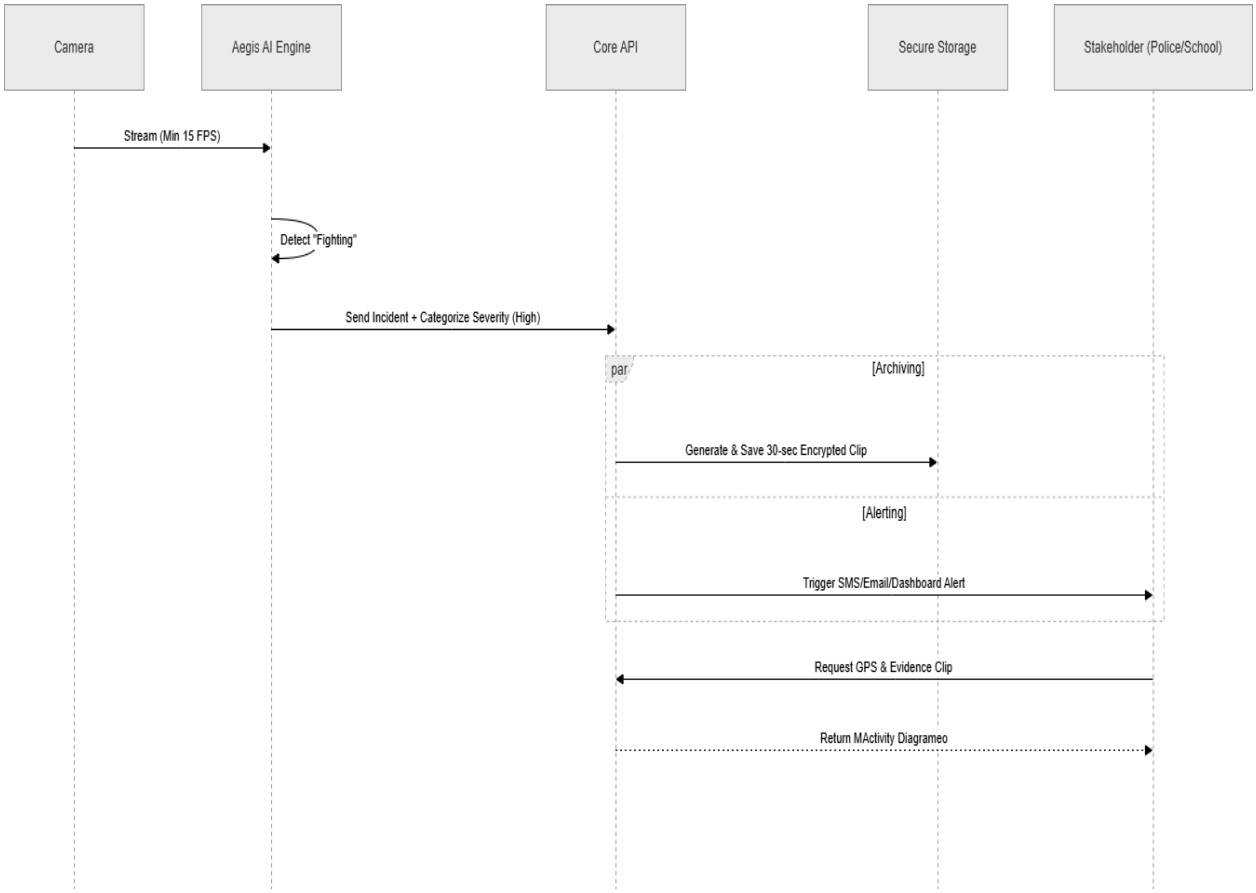
---

3. Data Flow & System Behavior

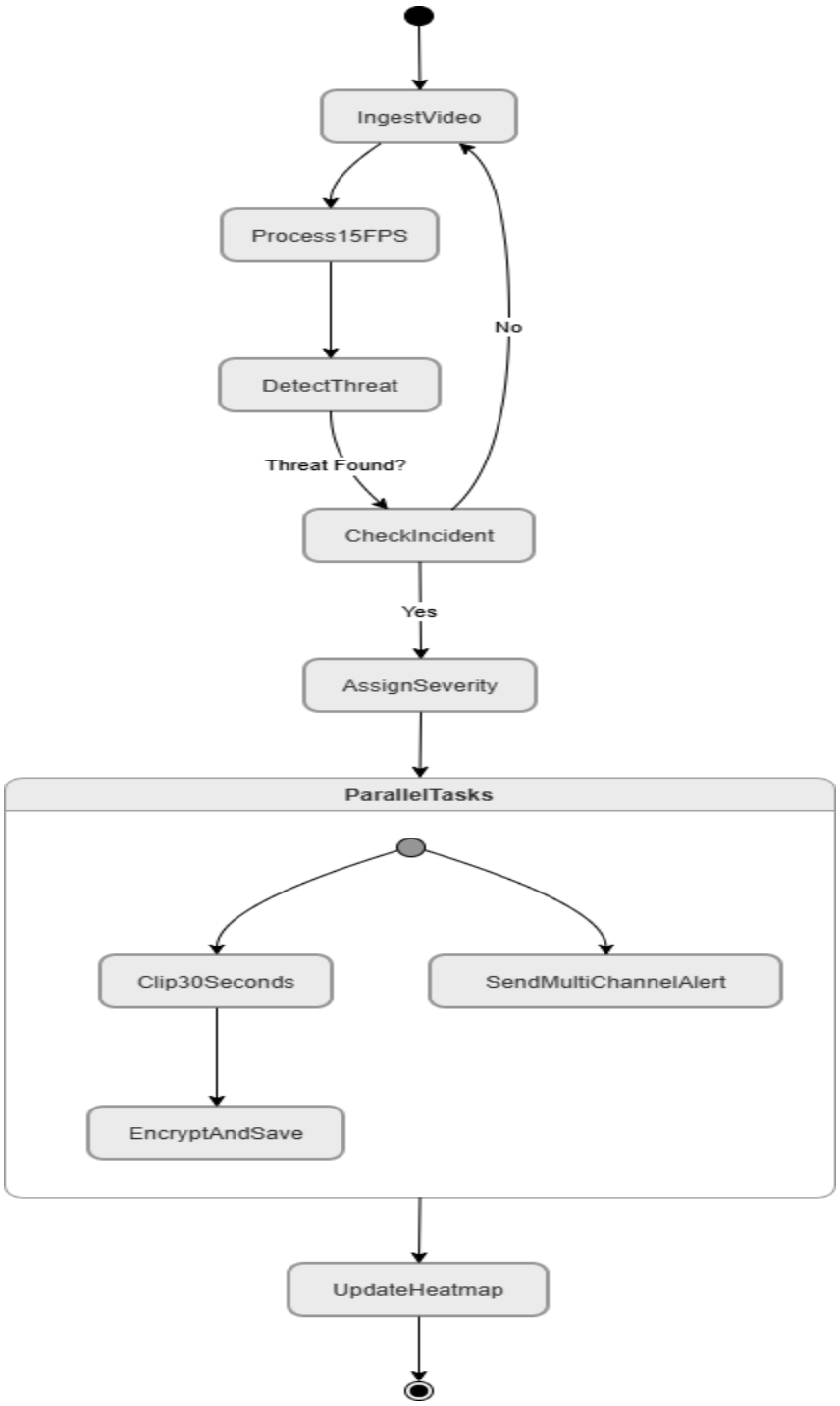
Context-Level Data Flow Diagram (DFD)



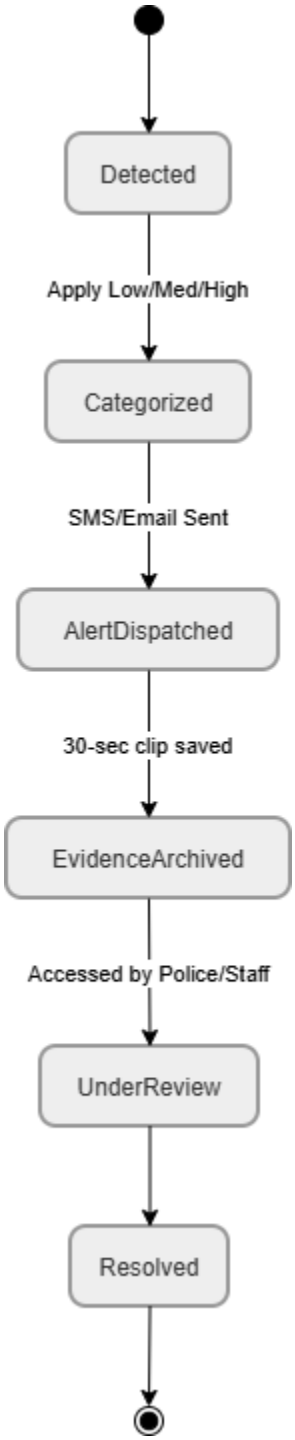
Sequence Diagram: Incident to Alert Workflow



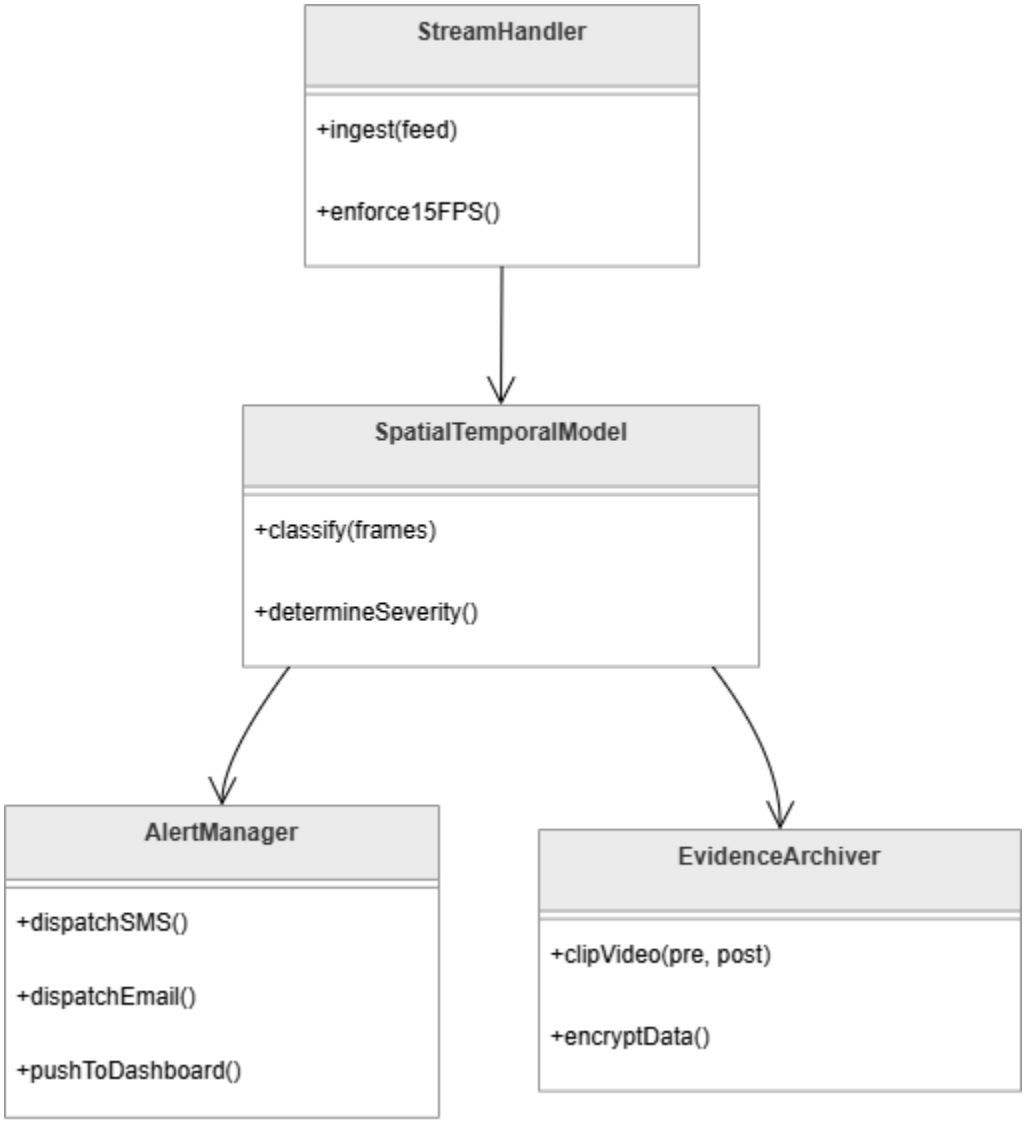
Activity Diagram



State Diagram (Incident Object)



Class Diagram



4. UI/UX Design & Prototyping

Wireframes & Mockups Strategy

- **Police/Emergency View:** A high-contrast GIS/Map-centric view showing precise real-time incident locations. Clicking a map pin instantly plays the 30-second evidence clip.
- **School Principal/SysAdmin View:** A centralized grid layout of cameras with a dedicated sidebar for "Frequency Heatmaps" (FR-4) to identify campus danger zones.
- **Parent View (Mobile Responsive):** A simplified, reassuring interface prioritizing child status verification and critical alerts.

UI/UX Guidelines

- **Color Scheme:** Neutral, dark-mode palettes to reduce eye strain, reserving strict color codes for severities: Yellow (Low), Orange (Medium), Red (High/Critical).
- **Typography:** Highly legible, sans-serif fonts for instant readability under stress.
- **Accessibility:** Cross-platform web UI compatible with desktop control rooms and mobile web browsers.

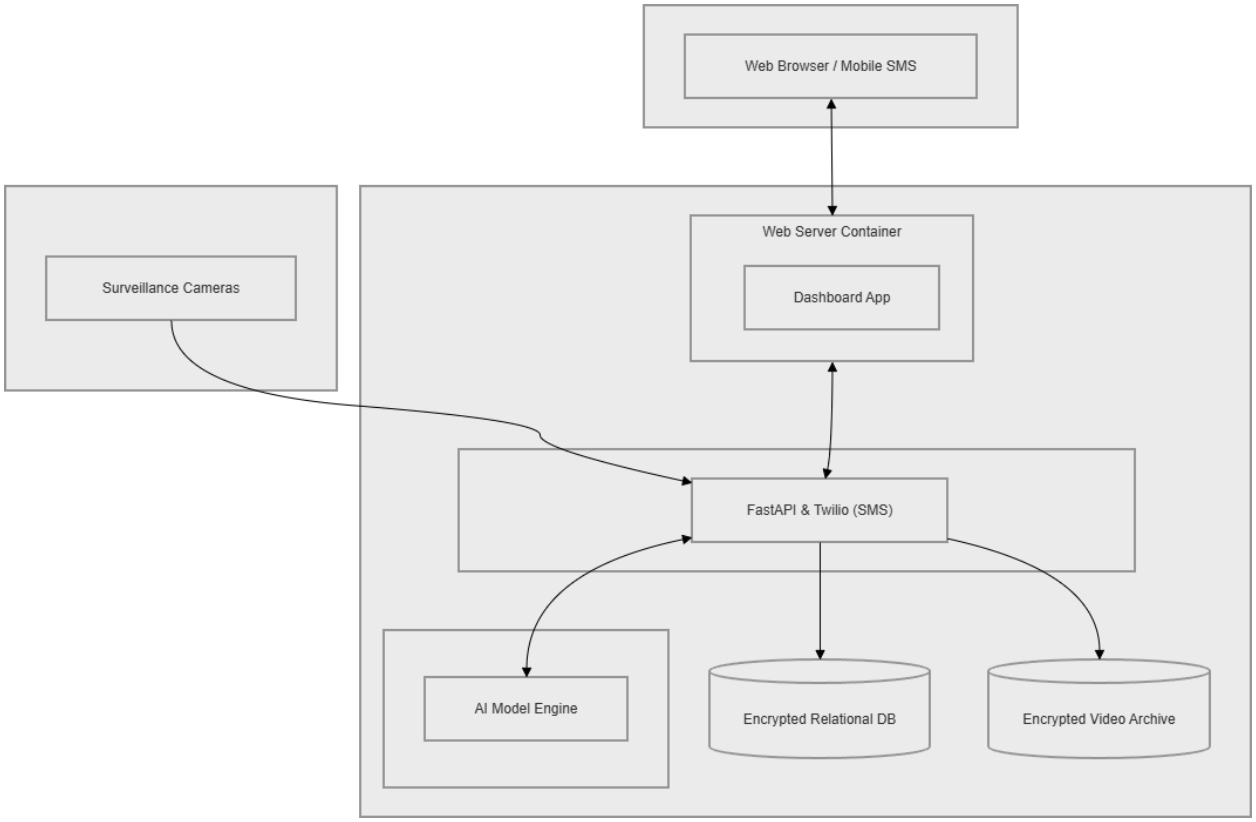
---

5. System Deployment & Integration

Technology Stack

- **Backend & APIs:** FastAPI / Flask (Python) to handle async video processing and API routing.
- **Frontend:** React.js or Vue.js for dynamic dashboard rendering and heatmaps.
- **AI Models:** PyTorch/TensorFlow utilizing Spatial-Temporal NNs.
- **Database & Storage:** PostgreSQL for relational data, AWS S3 / local encrypted blobs for the 30-second video clips.

Deployment Diagram



---

6. Additional Deliverables & Grading Criteria

- **Real-time Performance Testing:** Validate that the system reliably ingests feeds at a minimum of 15 FPS without bottlenecks.
- **Model Accuracy:** Run evaluation metrics against the test set to ensure high precision across the 13 defined incident types.
- **Integration Testing:** Ensure the API seamlessly connects the AI predictions, the automated 30-second clipping, and the alerting services.

Future Integration Vision (Literature Review Additions)

While focusing on the current scope, the architecture is designed to accommodate suggested improvements in future iterations:

- **GANs / IR Imaging** for advanced low-light detection.
- **Audio Integration** for multi-modal alerting (gunshots/screams).
- **Dedicated Mobile Application** for field response ecosystems.